Midterm Exam Study Guide

12 Topics • Bangla + English • Examples • Exam-Answer Structure

Quantum Computing • Blockchain • Cyber Security • LLM • Computer Vision • Ontology • Knowledge Representation • Semantic Web • CBR & Constraints • NLP • Quantum vs GenAI

এই guide টা ১ দিনে সব topic cover করার জন্য বানানো। প্রতিটা topic-এ: সহজ ব্যাখ্যা → key points → example/scenario → exam-এ কীভাবে লিখবে।

এক দিনে পড়ার Plan

One-Day Study Schedule

সময়	Topic	Focus
সকাল (৯০ মি.)	1. Quantum Computing, 12. Quantum vs GenAI	Qubit, Grover, Shor, তুলনার table
(৬০ মি.)	2. Blockchain, 3. Cyber Security	Block structure, consensus, CIA triad, attacks
দুপুর (৯০ মি.)	4, 5, 6, 11 (LLM, VLLM, CV, NLP)	Use-case, pipeline, GAN/Diffusion, Transformer
বিকাল (৯০ মি.)	7, 8, 9 (Ontology, KR, Frames, Semantic Net, Rule, Semantic Web)	Definition + example বেশি আসে
সন্ধ্যা (৪৫ মি.)	10. CBR & Constraints	4R cycle, map coloring, backtracking
রাত (৬০ মি.)	Revision	শুধু *Exam Tip* box আর শেষের Cheat Sheet

Exam-এ লেখার সাধারণ formula

১) Definition (১-২ লাইন) → ২) Key components/steps (point আকারে) → ৩) একটা real-life example → ৪) Advantage/limitation → ৫) Diagram/table (পারলে)। এই ৫ ধাপ মানলে যেকোনো প্রশ্নে full marks-এর কাছাকাছি যাওয়া যায়।

নোট

Topic 5-এর "VLLM" মানে আমি দুইভাবে cover করেছি: (a) Vision-Language Large Models (ছবি + text বুঝে), (b) vLLM inference engine। তোমার course-এ যেটা পড়ানো হয়েছে সেটা দেখে নিও।

1. Quantum Computing

Principles • Qubits • Grover's • Shor's

১.১ মূল ধারণা (Principles)

Quantum computing হলো quantum mechanics-এর নিয়ম (superposition, entanglement, interference) ব্যবহার করে computation করা। Classical computer-এ **bit** হয় 0 নয়তো 1; quantum computer-এ **qubit** একসাথে 0 ও 1-এর "মিশ্রণ" হতে পারে যতক্ষণ না মাপা (measure) হচ্ছে।

Principle	সহজ ব্যাখ্যা
Superposition	Qubit একসাথে |0⟩ ও |1⟩-এর combination। Coin ঘুরছে অবস্থায় head-tail দুটোই "সম্ভাবনা" হিসেবে আছে।
Entanglement	দুটো qubit এমনভাবে জড়িত যে একটার result জানলে অন্যটার result সাথে সাথে বোঝা যায়, দূরত্ব যতই হোক।
Interference	ভুল answer-এর amplitude কমিয়ে (cancel) আর সঠিক answer-এর amplitude বাড়িয়ে (constructive) দেওয়া। Quantum algorithm-এর আসল শক্তি এটাই।
Measurement	মাপলে superposition ভেঙে যায় (collapse), একটা definite 0 বা 1 পাওয়া যায়।

১.২ Qubit

একটা qubit-এর state: `|ψ⟩ = α|0⟩ + β|1⟩`, যেখানে α, β complex number এবং `|α|² + |β|² = 1`। মাপলে 0 পাওয়ার probability $|\alpha|^2$, 1 পাওয়ার probability $|\beta|^2$।

- **Bloch sphere:** একটা qubit-এর state sphere-এর উপর একটা বিন্দু হিসেবে দেখানো হয়। North pole = |0⟩, South pole = |1⟩।
- **n qubit** = একসাথে 2^n টা state-এর amplitude। 3 qubit = 8 state, 50 qubit ≈ 10^{15} state।
- **Gates:** X (NOT gate, 0↔1), H (Hadamard, superposition বানায়), Z (phase flip), CNOT (2-qubit, entanglement বানায়)।
- **Physical qubit:** superconducting circuit, trapped ion, photon ইত্যাদি। Noise/decoherence বড় সমস্যা, তাই error correction লাগে।

> **Example: Bell state (Entanglement)**
> |00⟩ থেকে শুরু → প্রথম qubit-এ H দাও → CNOT দাও। Result: `(|00⟩ + |11⟩)/√2`। মাপলে 50% সময় 00, 50% সময় 11 আসবে; কখনোই 01 বা 10 না। একটা qubit 0 পেলে অন্যটাও 0 হবেই। এটাই entanglement।

১.৩ Grover's Algorithm

কাজ: Unsorted database/search space-এ target item খোঁজা। Classical-এ N item-এ গড়ে ~N/2 চেষ্টা, worst case N; Grover-এ মাত্র **O(√N)** চেষ্টা। অর্থাৎ **quadratic speedup** (exponential না)।

1. **Initialize:** সব qubit-এ H দিয়ে সব N state-এর equal superposition বানাও।
2. **Oracle:** সঠিক answer-এর amplitude-এর sign উল্টে দেয় (phase flip)।
3. **Diffusion operator (amplitude amplification):** গড় amplitude-এর চারপাশে reflect করে, ফলে সঠিক answer-এর amplitude বাড়ে।
4. Step 2-3 প্রায় `(π/4)·√N` বার repeat করো, তারপর measure করো।

Example (N = 4, 2 qubit)
4টা item (00, 01, 10, 11), target = 10। শুরুতে প্রতিটার probability 25%। মাত্র **1 iteration**-এর পর 10 পাওয়ার probability **100%**। Classical-এ গড়ে 2-3 বার চেক করতে হতো।
Real scenario: 10 লাখ password-এর list থেকে একটা hash মেলানো: classical ~10^6 step, Grover ~10^3 step।

Limitation: শুধু quadratic speedup; oracle বানানো লাগে; সব problem-এ কাজ করে না। Symmetric crypto-র জন্য সমাধান: key length দ্বিগুণ করা (AES-128 → AES-256)।

১.৪ Shor's Algorithm

কাজ: বড় সংখ্যাকে prime factor-এ ভাঙা। Classical best algorithm sub-exponential; Shor's **polynomial time** (প্রায় $O((\log N)^3)$)। এজন্য **RSA, Diffie-Hellman, ECC** ভাঙার হুমকি।

মূল idea: Factoring-কে **period finding** সমস্যায় রূপান্তর করা। Period finding অংশ quantum (**Quantum Fourier Transform** দিয়ে), বাকিটা classical।

1. N-এর সাথে coprime একটা a বেছে নাও ($1 < a < N$)।
2. Function $f(x) = a^x \bmod N$-এর **period r** বের করো (quantum অংশ, QFT)।
3. যদি r জোড় এবং $a^{r/2} \not\equiv -1 \pmod N$ হয়, তাহলে $\gcd(a^{r/2} \pm 1, N)$ থেকে factor পাওয়া যায়।
4. না হলে অন্য a দিয়ে আবার চেষ্টা।

Example: N = 15 factor করা
a = 7 নিলাম। $7^1=7$, $7^2=49 \bmod 15=4$, $7^3=28 \bmod 15=13$, $7^4=91 \bmod 15=1$ → **period r = 4** (জোড়)। $a^{r/2} = 7^2 = 49$। $\gcd(49-1, 15) = \gcd(48, 15) = \mathbf{3}$; $\gcd(49+1, 15) = \gcd(50, 15) = \mathbf{5}$। অতএব $15 = 3 \times 5$।
Quantum computer-এর কাজ শুধু r = 4 দ্রুত বের করা; ছোট সংখ্যায় আমরা হাতে করছি, কিন্তু 2048-bit সংখ্যায় classical-এ এটা অসম্ভব।

	Grover	Shor
সমস্যা	Search (unstructured)	Integer factoring / discrete log
Speedup	Quadratic (N → √N)	Exponential (super-polynomial)
মূল technique	Amplitude amplification	Period finding + QFT
প্রভাব	Symmetric key দুর্বল (key size দ্বিগুণ করলেই চলে)	RSA/ECC সম্পূর্ণ ভাঙে → Post-Quantum Cryptography দরকার

Exam Tip
"Explain Grover's algorithm" এলে: purpose → 3 steps (superposition, oracle, diffusion) → O(√N) → N=4 example। "Shor's" এলে: factoring → period finding → N=15, a=7 example → RSA-এর ওপর প্রভাব। দুটোর তুলনার table লিখলে বাড়তি নম্বর।

2. Blockchain

Distributed ledger • Consensus • Smart contracts

২.১ সংজ্ঞা

Blockchain হলো একটা **distributed, decentralized, immutable ledger**, যেখানে transaction গুলো **block**-এ জমা হয় এবং প্রতিটা block আগের block-এর **hash** দিয়ে cryptographically জোড়া থাকে। কোনো একক

কর্তৃপক্ষ (যেমন ব্যাংক) লাগে না।

২.২ Block-এর গঠন

Field	কাজ
Previous hash	আগের block-এর fingerprint, এটাই "chain" বানায়
Merkle root	block-এর সব transaction-এর hash tree-র root; একটা transaction বদলালে root বদলে যায়
Timestamp, Nonce	সময়; Nonce = Proof-of-Work-এ মেলানোর জন্য বদলানো সংখ্যা
Transactions	আসল data (কে কাকে কত পাঠাল)
Block hash	পুরো header-এর SHA-256 hash

কেন tamper করা যায় না?
Block 5 এর data বদলালে Block 5 এর hash বদলাবে → Block 6 এর "previous hash" মিলবে না → Block 6, 7, 8... সবগুলো আবার হিসাব করতে হবে, এবং network-এর বেশিরভাগ node-কে convince করতে হবে। এটা প্রায় অসম্ভব।

২.৩ Consensus Mechanism

Type	কীভাবে কাজ করে	সুবিধা / অসুবিধা
Proof of Work (PoW)	Miner রা hash puzzle সমাধান করে (nonce খুঁজে, hash-এর শুরুতে নির্দিষ্ট সংখ্যক 0)। যে আগে পারে সে block যোগ করে ও reward পায়। Bitcoin ব্যবহার করে।	খুব নিরাপদ / প্রচুর বিদ্যুৎ, ধীর (~7 TPS)
Proof of Stake (PoS)	Validator কত coin "stake" করেছে তার ভিত্তিতে block validate করার সুযোগ পায়। ঠকালে stake কাটা যায়। Ethereum এখন এটা ব্যবহার করে।	কম energy, দ্রুত / ধনী validator-এর প্রভাব বেশি
PBFT / others	Permissioned network-এ node রা ভোট করে	দ্রুত / node সংখ্যা সীমিত

২.৪ প্রকারভেদ ও Smart Contract

- **Public** (Bitcoin, Ethereum): যে কেউ join করতে পারে। **Private/Consortium** (Hyperledger Fabric): শুধু অনুমোদিত সদস্য, কোম্পানিগুলো ব্যবহার করে।
- **Smart contract:** blockchain-এ থাকা self-executing code। শর্ত পূরণ হলে নিজে নিজে চলে, মাঝে তৃতীয় পক্ষ লাগে না।
- **Key properties:** Decentralization, Transparency, Immutability, Security (cryptography), Traceability।

Example 1: Land Registry (জমির দলিল)
জমির মালিকানা বদলের প্রতিটা record blockchain-এ থাকে। একই জমি দুইবার বিক্রি (double-selling) বা জাল দলিল ধরা সহজ, কারণ আগের সব record বদলানো যায় না। Smart contract: "ক্রেতা টাকা দিলে স্বয়ংক্রিয়ভাবে মালিকানা তার নামে হবে।"
Example 2: Supply chain: আম বাগান → প্যাকিং → পরিবহন → দোকান; প্রতিধাপে scan করে blockchain-এ record। ক্রেতা QR scan করে জানে আম কোথাকার।

Challenges: Scalability (কম TPS), energy use (PoW), 51% attack (কেউ network-এর 50%+ computing power নিলে), privacy, regulation।

Exam Tip
Diagram আঁকো: [Block 1 | hash H1] → [Block 2 | prev H1, hash H2] → [Block 3 | prev H2, ...]। এটা আঁকলেই অর্ধেক নম্বর নিশ্চিত। তারপর PoW vs PoS table।

3. Cyber Security

CIA triad • Threats • Defense

৩.১ সংজ্ঞা ও CIA Triad

Cyber security হলো computer, network, data ও সিস্টেমকে অননুমোদিত access, আক্রমণ ও ক্ষতি থেকে রক্ষা করার অনুশীলন।

Principle	অর্থ	Threat-এর উদাহরণ	রক্ষা
Confidentiality	শুধু অনুমোদিত ব্যক্তিই data দেখবে	Data breach, sniffing	Encryption, access control
Integrity	Data অবিকৃত থাকবে	Data tampering, MITM	Hashing, digital signature
Availability	দরকারের সময় service পাওয়া যাবে	DDoS, ransomware	Backup, redundancy, firewall

৩.২ প্রধান Threat ও Attack

Attack	কী	উদাহরণ
Malware (virus, worm, trojan, ransomware, spyware)	ক্ষতিকর software	Ransomware file encrypt করে টাকা চায়
Phishing	ভুয়া email/site দিয়ে password/ টাকা হাতানো	"আপনার ব্যাংক একাউন্ট বন্ধ হবে, এখানে login করুন" লিঙ্ক
SQL Injection	Input field-এ SQL code ঢুকিয়ে database নিয়ন্ত্রণ	Password field-এ `' OR '1'='1` দিলে login bypass
XSS	Website-এ malicious script ঢোকানো	Comment box-এ script দিয়ে অন্যের cookie চুরি
DoS / DDoS	প্রচুর request পাঠিয়ে server অচল করা	হাজারো botnet একসাথে একটা site-এ hit
Man-in-the-Middle	দুই পক্ষের মাঝখানে বসে communication শোনা/বদলানো	Public WiFi-তে data ধরা
Social engineering	মানুষকে ঠকিয়ে তথ্য নেওয়া	IT staff সেজে ফোনে password চাওয়া

৩.৩ Defense Tools

- **Cryptography:** Symmetric (AES, একই key) দ্রুত; Asymmetric (RSA, public+private key) key বিনিময়ে; Hash (SHA-256) integrity যাচাইয়ে; Digital signature = hash + private key।
- **Authentication:** Password + **MFA/2FA** (OTP, biometric)।
- **Firewall** (traffic filter), **IDS/IPS** (আক্রমণ detect/prevent), **VPN**, **antivirus**, regular **patching**।
- **Defense in depth:** একাধিক স্তরে নিরাপত্তা (network → host → application → data → মানুষ/training)।
- **Prevention:** SQLi-র জন্য parameterized query; XSS-এর জন্য input sanitization।

Scenario: ব্যাংকিং phishing
রহিম একটা SMS পেল "আপনার কার্ড block হয়েছে, bkash-bank-verify.xyz এ যান"। সে link-এ গিয়ে card number ও OTP দিল → attacker টাকা তুলে নিল। **Violated:** Confidentiality। **Prevention:** URL যাচাই, OTP কাউকে না দেওয়া, MFA, bank-এর official app ব্যবহার, user awareness training।

Exam Tip
CIA triad সবসময় লিখো। Attack বর্ণনায় "কী → কীভাবে কাজ করে → একটা example → প্রতিরোধ" এই চার ধাপ। SQL injection-এর query example লিখলে স্পষ্ট impression পড়ে।

4. LLM in Software Engineering

Code generation • Testing • Review • Risks

৪.১ LLM কী?

Large Language Model (LLM) হলো Transformer-ভিত্তিক বিশাল neural network, যা বিপুল text ও code দিয়ে train করা এবং পরবর্তী token predict করে text/code তৈরি করে। উদাহরণ: GPT, Claude, Gemini, Llama, Code Llama, StarCoder।

৪.২ Software Development Life Cycle (SDLC)-এ ব্যবহার

Phase	LLM কী করে
Requirements	ক্লায়েন্টের অগোছালো বর্ণনা থেকে user story, SRS draft; ambiguity ধরা
Design	Architecture suggestion, UML/ER diagram description, API design
Coding	Code generation, auto-completion (GitHub Copilot), এক language থেকে আরেকটায় translation
Testing	Unit test case তৈরি, edge case খোঁজা, test data generation
Debugging	Error message ব্যাখ্যা, bug ও fix প্রস্তাব, **automated program repair**
Review & Docs	Code review comment, docstring/README, commit message লেখা
Maintenance	Legacy code ব্যাখ্যা, refactoring, code smell শনাক্ত

৪.৩ Key Techniques

- **Prompt engineering:** স্পষ্ট context, example (few-shot), step-by-step (chain-of-thought)।
- **RAG (Retrieval-Augmented Generation):** নিজের codebase/docs থেকে প্রাসঙ্গিক অংশ খুঁজে prompt-এ দেওয়া, ফলে hallucination কমে।
- **Fine-tuning:** কোম্পানির নিজস্ব code style/domain-এ অভিযোজন।
- **Agents:** LLM নিজে tool চালায় (file edit, test run, git), ভুল হলে নিজে ঠিক করে।
- **Evaluation:** HumanEval, MBPP benchmark; metric **pass@k** = k বার চেষ্টায় অন্তত একটা সঠিক code হওয়ার সম্ভাবনা।

Example: Test generation
Prompt: "নিচের function-এর জন্য pytest unit test লেখো, edge case সহ।" Function: `def divide(a,b): return a/b`। LLM test বানায়: normal (10/2), negative, float, এবং **b=0 হলে ZeroDivisionError** expect করা test। Developer-এর ঘণ্টাখানেকের কাজ কয়েক সেকেন্ডে, কিন্তু human কে review করতে হয়।

8.8 Risks & Limitations

- **Hallucination:** অস্তিত্বহীন function/library বানিয়ে ফেলা।
- **Security:** vulnerable code (SQLi, hard-coded secret), prompt injection।
- **License/IP:** training data থেকে copyrighted code হুবহু আসতে পারে।
- **Over-reliance:** developer skill কমে, ভুল code না বুঝে merge।
- **Privacy:** proprietary code বাইরের API-তে পাঠানো।
- বড় project-এর context window সীমা।

> **Exam Tip**
> উত্তরে SDLC phase অনুযায়ী table + একটা example (test generation বা bug fix) + benefits vs risks দুই কলাম। শেষে conclusion: "LLM developer-কে replace করে না, productivity বাড়ায়; human-in-the-loop জরুরি।"

5. LLM & VLLM in Data Science

Text-to-SQL • EDA • Vision-Language Models • vLLM serving

৫.১ Data Science pipeline-এ LLM

ধাপ	LLM-এর ভূমিকা
Data collection	Unstructured text (review, PDF, email) থেকে structured field বের করা (information extraction)
Cleaning	বানান/format ঠিক করা, duplicate ও missing value-র মান অনুমান, category standardize ("Dhaka", "ঢাকা", "DHK" → Dhaka)
EDA	Natural language-এ প্রশ্ন করলে pandas/SQL code ও chart বানিয়ে দেয়
Feature engineering	Text embedding, নতুন feature-এর idea
Modeling	Model suggestion, hyperparameter ও code boilerplate
Reporting	Result ব্যাখ্যা, summary, insight narrative

> **Example: Text-to-SQL**
> প্রশ্ন: "গত মাসে কোন ৫টা product সবচেয়ে বেশি বিক্রি হয়েছে?" → LLM schema দেখে বানায়:
>
> ```
> SELECT product_name, SUM(qty) AS total
> FROM sales
> WHERE sale_date >= DATE_SUB(CURDATE(), INTERVAL 1 MONTH)
> GROUP BY product_name ORDER BY total DESC LIMIT 5;
> ```
>
> Business user SQL না জেনেও data থেকে উত্তর পায়। **সাবধানতা:** query চালানোর আগে যাচাই (ভুল JOIN, ভুল column)।

৫.২ VLLM: দুটো অর্থ

(a) Vision-Language (Large) Models — VLM

এই model একসাথে **ছবি + text** বোঝে। গঠন: Vision encoder (ViT/CLIP) → projection layer → LLM। উদাহরণ: GPT-4V/4o, Gemini, Claude (vision), LLaVA, CLIP।

- **Chart/graph বোঝা:** dashboard-এর ছবি দিলে trend ব্যাখ্যা করে।

- **Document/OCR:** invoice, form, table-এর ছবি থেকে data extraction।
- **Medical/Satellite image** বর্ণনা ও প্রশ্নোত্তর (Visual Question Answering)।
- **Image captioning, image-text retrieval**, multimodal dataset labeling।

> **Scenario**
> একটা কোম্পানির হাজার হাজার হাতে লেখা/scan করা বিলের ছবি আছে। VLM-কে বলা হলো: "এই ছবি থেকে date, vendor, total JSON-এ দাও।" ফলে manual data entry ছাড়াই CSV তৈরি → তারপর সাধারণ data science analysis।

(b) vLLM — inference/serving engine

vLLM হলো open-source library, যা LLM দ্রুত ও কম memory-তে serve করে। মূল innovation **PagedAttention**: KV cache-কে OS-এর virtual memory paging-এর মতো ছোট block-এ ভাগ করে, ফলে GPU memory অপচয় কমে। সাথে **continuous batching** → বেশি throughput। Data science team যখন নিজেদের LLM production-এ চালায় তখন কাজে লাগে।

> **Exam Tip**
> "VLLM" নিয়ে প্রশ্ন এলে আগে এক লাইনে ধরে নাও কোনটা (প্রশ্নের context দেখে)। দ্বিধা থাকলে দুটোই সংক্ষেপে লিখো। LLM-এর limitation (hallucination, data privacy, ভুল SQL) অবশ্যই উল্লেখ করো।

6. Computer Vision Algorithms & Generative AI

Classical CV • CNN • Detection • GAN • Diffusion

৬.১ Computer Vision (CV) কী?

কম্পিউটারকে ছবি/ভিডিও থেকে অর্থ বুঝতে শেখানো। মূল task: **Classification** (ছবিতে কী আছে), **Detection** (কোথায় আছে, bounding box), **Segmentation** (pixel-level), Tracking, Pose estimation, OCR।

৬.২ Classical algorithms

Algorithm	কাজ
Sobel / **Canny**	Edge detection (Canny: blur → gradient → non-max suppression → threshold)
Harris corner	Corner শনাক্ত
SIFT / SURF / ORB	Scale/rotation-invariant feature, image matching, panorama stitching
HOG + SVM	Pedestrian detection
Hough transform	Line/circle শনাক্ত
K-means, Watershed	Segmentation

৬.৩ Deep Learning

- **CNN:** Convolution layer (filter দিয়ে feature যেমন edge → shape → object) → Pooling (ছোট করা) → Fully connected → Softmax। উদাহরণ: LeNet, AlexNet, VGG, **ResNet** (skip connection দিয়ে vanishing gradient সমস্যা সমাধান)।
- **Object detection:** R-CNN/Faster R-CNN (দুই ধাপ, নির্ভুল), **YOLO** / SSD (এক ধাপ, real-time)।
- **Segmentation:** U-Net (medical), Mask R-CNN।
- **Vision Transformer (ViT):** ছবিকে patch-এ ভেঙে Transformer-এ দেওয়া।

Example: Self-driving car
Camera frame → YOLO দিয়ে গাড়ি/মানুষ/traffic light detect → U-Net দিয়ে রাস্তা segment → lane detection (Canny + Hough) → সিদ্ধান্ত: ব্রেক/টার্ন।

৬.৪ Generative AI in Vision

Model	ধারণা	সুবিধা / সমস্যা
GAN	দুটো network প্রতিযোগিতা করে: **Generator** নকল ছবি বানায়, **Discriminator** আসল-নকল ধরে। দুজনই উন্নত হয়। (জাল নোট বানানো বনাম পুলিশ)	তীক্ষ্ণ ছবি / training অস্থির, mode collapse
VAE	Encoder ছবিকে latent space-এ নেয়, decoder সেখান থেকে ছবি বানায়	Stable / ছবি কিছুটা ঝাপসা
Diffusion	Forward: ছবিতে ধাপে ধাপে noise মেশানো। Reverse: noise থেকে ধাপে ধাপে noise সরিয়ে ছবি বানানো (text prompt দিয়ে নিয়ন্ত্রিত)। Stable Diffusion, DALL·E এভাবে কাজ করে।	সেরা মান ও বৈচিত্র্য / ধীর (অনেক step)

Use cases: text-to-image, image inpainting, super-resolution, style transfer, data augmentation (medical image কম থাকলে synthetic বানানো), deepfake (ঝুঁকি)।

Exam Tip
GAN-এর generator/discriminator diagram আঁকো। CNN-এর layer flow লিখো। "Discriminative vs Generative" পার্থক্যও আসে: discriminative P(y|x) শেখে (classify), generative P(x) শেখে (নতুন data বানায়)।

7. Ontologies & Data Mining

Concepts • OWL • KDD • Apriori • Clustering

৭.১ Ontology

Ontology হলো কোনো domain-এর concept, তাদের সম্পর্ক ও নিয়মের একটা **formal, explicit, shared specification** (Gruber: "explicit specification of a conceptualization")। সহজে: একটা বিষয়ের "শব্দকোষ + সম্পর্কের মানচিত্র" যা machine ও মানুষ দুজনেই বোঝে।

উপাদান	অর্থ	উদাহরণ (University)
Classes (Concepts)	জিনিসের শ্রেণি	Person, Student, Professor, Course
Instances (Individuals)	নির্দিষ্ট বস্তু	Rahim (Student), CSE101 (Course)
Properties (Relations)	সম্পর্ক/বৈশিষ্ট্য	enrolledIn, teaches, hasEmail
Axioms / Rules	নিয়ম	Student ⊑ Person; একজন Student কমপক্ষে ১টা Course-এ enrolled

ভাষা: RDF → RDFS → **OWL** (Web Ontology Language), tool: Protégé। Reasoner (HermiT) নতুন জ্ঞান অনুমান করে। **Use:** medical (SNOMED, Gene Ontology), e-commerce product taxonomy, knowledge graph।

Reasoning example
Ontology: Professor ⊑ Person; Rahim একজন Professor। কেউ কখনো লেখেনি "Rahim is a Person", কিন্তু reasoner অনুমান করে Rahim ∈ Person।

৭.২ Data Mining

Data mining হলো বিপুল data থেকে আগে অজানা, দরকারি pattern ও জ্ঞান আবিষ্কার। এটা **KDD** (Knowledge Discovery in Databases) প্রক্রিয়ার মূল ধাপ: Selection → Preprocessing/Cleaning → Transformation → **Data Mining** → Interpretation/Evaluation।

Task	কাজ	Algorithm	উদাহরণ
Classification	Label predict (supervised)	Decision Tree, Naive Bayes, SVM, kNN	Loan approve/reject
Clustering	অনুরূপ গ্রুপ খোঁজা (unsupervised)	K-means, DBSCAN, Hierarchical	Customer segmentation
Association rule	একসাথে ঘটা জিনিস খোঁজা	Apriori, FP-Growth	Market basket
Regression	সংখ্যা predict	Linear regression	বাড়ির দাম
Anomaly detection	ব্যতিক্রম ধরা	Isolation Forest	Credit card fraud

Apriori numeric example
৫টা transaction: T1{bread, milk}, T2{bread, diaper, beer}, T3{milk, diaper, beer, cola}, T4{bread, milk, diaper, beer}, T5{bread, milk, diaper, cola}।
Rule **{diaper} → {beer}**: diaper আছে ৪টায় (T2,T3,T4,T5), diaper+beer আছে ৩টায় (T2,T3,T4)।
Support = 3/5 = 60%। **Confidence** = 3/4 = 75%। **Lift** = conf / P(beer) = 0.75 / 0.6 = 1.25 (>1 মানে ইতিবাচক সম্পর্ক)।
Support = কতবার একসাথে এসেছে; Confidence = A থাকলে B থাকার সম্ভাবনা।

৭.৩ Ontology + Data Mining

Ontology data mining-কে **semantic** করে: যেমন "Apple" শব্দটা ফল না কোম্পানি, ontology দিয়ে ঠিক হয়; mining result-কে concept hierarchy-তে সাধারণীকরণ (Coke, Pepsi → Soft drink) করে multi-level association rule পাওয়া যায়। উল্টো দিকে data mining থেকে ontology-তে নতুন concept শেখা (**ontology learning**)।

Exam Tip
Ontology-তে একটা ছোট university বা hospital ontology আঁকো (Class-Subclass তীর সহ)। Data mining-এ KDD flow + Apriori-র support/confidence হিসাব দেখাতে পারলে full marks সম্ভাবনা বেশি।

8. Knowledge Representation, Frame-Based Systems & Representations for KBS

KR schemes • Frames • Slots • Inheritance

৮.১ Knowledge Representation (KR)

KR হলো বাস্তব জগতের জ্ঞানকে এমন আকারে সাজানো যাতে কম্পিউটার তা সংরক্ষণ করতে ও তা থেকে যুক্তি (reasoning) করে নতুন সিদ্ধান্ত বের করতে পারে। **KBS (Knowledge-Based System)** = Knowledge base + Inference engine (+ user interface)।

ভালো KR-এর ৪টা বৈশিষ্ট্য

Property	অর্থ
Representational Adequacy	প্রয়োজনীয় সব ধরনের জ্ঞান প্রকাশ করতে পারা
Inferential Adequacy	পুরোনো জ্ঞান থেকে নতুন জ্ঞান বের করার ক্ষমতা
Inferential Efficiency	দ্রুত ও কম resource-এ reasoning
Acquisitional Efficiency	নতুন জ্ঞান সহজে যোগ করা যায়

৮.২ KR-এর প্রধান পদ্ধতি

পদ্ধতি	সংক্ষেপে	উদাহরণ
Propositional / Predicate Logic	সূত্রের মাধ্যমে নির্ভুল জ্ঞান	∀x Student(x) → Person(x)
Production Rules	IF–THEN	IF জ্বর AND কাশি THEN ফ্লু সম্ভব
Semantic Networks	Node ও labeled arc-এর graph	Bird –is-a→ Animal
Frames	Object-এর structured record	নিচে বিস্তারিত
Scripts	ঘটনার ধারাবাহিক ক্রম	রেস্টুরেন্টে খাওয়া: ঢোকা→অর্ডার→খাওয়া→বিল→বের হওয়া
Ontologies / Description Logic	Formal concept hierarchy	OWL

৮.৩ Frame-Based Systems

Frame (Marvin Minsky, 1975) হলো কোনো object/concept-এর stereotypical জ্ঞান ধরে রাখার data structure। একটা frame-এ থাকে **slots** (attribute), প্রতিটা slot-এ থাকে **facets** (value, default, range/type, procedure)।

- **Inheritance:** Child frame, parent frame-এর slot উত্তরাধিকার সূত্রে পায় (`is-a`, `instance-of` link দিয়ে)।
- **Default value:** কিছু না জানা থাকলে ধরে নেওয়া মান (Bird.can_fly = true)। Child এ override করা যায় (Penguin.can_fly = false)।
- **Procedural attachment (Demons):** *if-needed* (দরকার হলে হিসাব), *if-added* (মান বসলে trigger), *if-removed*।

Example: Vehicle frame hierarchy

```
Frame: Vehicle
  wheels: 4 (default)
  fuel: petrol (default)

Frame: Car        (is-a Vehicle)
  seats: 5
  max_speed: if-needed -> calculate_from(engine_cc)

Frame: MyToyota   (instance-of Car)
  color: white
  engine_cc: 1500
  owner: Karim
```

প্রশ্ন: MyToyota-র wheels কয়টা? → MyToyota-তে নেই → Car-এ নেই → Vehicle থেকে inherit: **4**। max_speed চাইলে if-needed demon চলে engine_cc থেকে হিসাব করে।

সুবিধা	অসুবিধা
Natural organization, inheritance-এ redundancy কম, default reasoning, procedure জুড়ে দেওয়া যায়	Exception handling জটিল (multiple inheritance conflict), কঠোর formal semantics নেই, জটিল logic reasoning দুর্বল

৮.৪ Representations for KBS

KBS বানাতে সমস্যা অনুযায়ী representation বাছতে হয়: **Rule-based** (diagnosis, expert system যেমন MYCIN), **Frame-based** (structured object জ্ঞান, যেমন product/patient record), **Logic** (নিশ্চিত প্রমাণ), **Semantic net/Ontology** (concept সম্পর্ক)। বাস্তবে প্রায়ই **hybrid** (frame + rule) ব্যবহার হয়।

Exam Tip
Frame-এর প্রশ্নে সবসময় একটা hierarchy example আঁকো ও inheritance দিয়ে একটা query সমাধান করে দেখাও। 4টা KR property মুখস্থ রাখো (RA, IA, IE, AE)।

9. Semantic Networks, Rule-Based Systems & Semantic Web

Graph knowledge • Chaining • RDF/OWL/SPARQL

৯.১ Semantic Network

জ্ঞানকে **graph** আকারে দেখানো: **Node** = concept/object, **labeled arc** = সম্পর্ক। প্রধান link: `is-a` (শ্রেণি), `has-a / part-of`, `instance-of`।

Example

```
Tweety --instance-of--> Canary --is-a--> Bird --is-a--> Animal
Bird --has--> Wings        Bird --can--> Fly
Canary --color--> Yellow
```

প্রশ্ন: Tweety কি উড়তে পারে? Graph ধরে উপরে গিয়ে Bird-এ "can fly" পাওয়া যায় → **হ্যাঁ** (property inheritance)। **Limit:** exception (Penguin) ও quantifier (সব/কিছু) প্রকাশ কঠিন, standard semantics নেই।

৯.২ Rule-Based System

IF (condition) THEN (action/conclusion) আকারে জ্ঞান। Expert system-এর মূল।

অংশ	কাজ
Knowledge base (rules)	সব নিয়ম
Working memory	বর্তমান fact
Inference engine	Rule মিলিয়ে conclusion বের করে
Explanation facility	"কেন এই সিদ্ধান্ত?" ব্যাখ্যা

Forward chaining (Data-driven)	Backward chaining (Goal-driven)
Fact থেকে শুরু → rule fire করে নতুন fact → লক্ষ্যে পৌঁছানো	Goal (hypothesis) থেকে শুরু → কোন rule এটা প্রমাণ করবে → তার শর্ত প্রমাণ করা
Monitoring, planning-এ ভালো	Diagnosis-এ ভালো (MYCIN)

Example: Chaining
Rules: R1: IF A AND B THEN C. R2: IF C THEN D. R3: IF D AND E THEN F। Facts: A, B, E। Goal: F?
Forward: A,B → C (R1) → D (R2) → D,E → F (R3) ✔।
Backward: F চাই → R3 দরকার D ও E → E fact ✔; D চাই → R2 দরকার C → R1 দরকার A,B ✔ → F প্রমাণিত।
Conflict resolution (একাধিক rule fire-যোগ্য হলে): priority, specificity (বেশি নির্দিষ্ট rule আগে), recency।

সুবিধা: বোঝা সহজ, modular, explanation দেওয়া যায়। **অসুবিধা:** rule অনেক হলে maintain কঠিন, uncertainty ভালো ধরে না (certainty factor/fuzzy লাগে), learning নেই।

৯.৩ Semantic Web

Tim Berners-Lee-র ধারণা: web-এর data এমনভাবে অর্থসহ (machine-understandable) করা যাতে software নিজে data সংযুক্ত করে বুঝতে পারে। বর্তমান web মানুষের জন্য, semantic web মেশিনের জন্যও।

- **URI/IRI:** প্রতিটা জিনিসের অনন্য পরিচয়।
- **RDF:** data **triple** আকারে: **Subject – Predicate – Object**।
- **RDFS:** class ও property hierarchy। **OWL:** আরও সমৃদ্ধ ontology (cardinality, disjoint, inverse)।
- **SPARQL:** RDF data-র query language। **Linked Data**, DBpedia, schema.org।

RDF triple + SPARQL

```
:Rahim   :studiesAt   :DhakaUniversity .
:Rahim   :age         22 .
:DhakaUniversity :locatedIn :Dhaka .

SELECT ?name WHERE {
  ?name :studiesAt ?u .
  ?u    :locatedIn :Dhaka .
}
```

উত্তর: Rahim। দুটো আলাদা triple জুড়ে (graph join) উত্তর এসেছে।

Layer cake (নিচ থেকে উপর): Unicode/URI → XML → RDF → RDFS → Ontology (OWL) → Rules (SWRL) → Logic → Proof → Trust।

> **Exam Tip**
> Forward vs backward chaining-এ ছোট rule set নিয়ে দুটোই ধাপে ধাপে দেখাও। Semantic web-এ layer cake আঁকা ও একটা RDF triple লেখা প্রায় সবসময় নম্বর আনে।

10. Case-Based Reasoning & Reasoning with Constraints

4R cycle • Similarity • CSP • Backtracking

১০.১ Case-Based Reasoning (CBR)

নতুন সমস্যা সমাধানে **অতীতের অনুরূপ সমস্যার সমাধান** ব্যবহার করা, ডাক্তার যেমন আগের রোগীর অভিজ্ঞতা কাজে লাগান। একটা **case** = Problem description + Solution (+ Outcome)।

CBR Cycle (4R)

1. **Retrieve:** case base থেকে সবচেয়ে অনুরূপ case খোঁজা (similarity measure, যেমন nearest neighbor)।
2. **Reuse:** সেই case-এর সমাধান নতুন সমস্যায় প্রয়োগ।
3. **Revise:** দরকারে সমাধান মানিয়ে নেওয়া/ঠিক করা (adaptation) ও যাচাই।
4. **Retain:** সফল নতুন case টা case base-এ রেখে দেওয়া (শেখা)।

Similarity: `Sim(new, old) = Σ wᵢ · simᵢ(featureᵢ)` (weighted sum)। Weight দিয়ে গুরুত্বপূর্ণ feature-কে বেশি মূল্য।

> **Example: IT Help Desk**
> নতুন সমস্যা: "Laptop WiFi connect হচ্ছে না, Windows 11, Intel adapter"। Case base-এ আগের case: "WiFi কাজ করছে না, Windows 11, Intel adapter → driver update করে ঠিক হয়েছে" (similarity 92%)। **Retrieve** করল, **Reuse**: driver update করতে বলল। কাজ না করলে **Revise**: adapter reset যোগ করা। সফল হলে **Retain**: নতুন case সংরক্ষণ।

সুবিধা: বিশেষজ্ঞের rule লিখতে হয় না, ধীরে ধীরে শেখে, ব্যাখ্যা সহজ। **অসুবিধা:** case base বড় হলে retrieval ধীর, ভালো similarity/adaptation বানানো কঠিন, নতুন ধরনের সমস্যায় দুর্বল। **Use:** help desk, law (precedent), medical diagnosis, recipe planning।

১০.২ Reasoning with Constraints (CSP)

Constraint Satisfaction Problem = **Variables (X)** + প্রতিটার **Domain (D)** + **Constraints (C)**। লক্ষ্য: সব constraint মেনে সব variable-এ মান বসানো।

> **Example: Map Coloring (Australia)**
> Variables: WA, NT, SA, Q, NSW, V, T। Domain: {Red, Green, Blue}। Constraint: প্রতিবেশী অঞ্চলের রং আলাদা (WA≠NT, WA≠SA, NT≠SA, SA≠Q, ...)।
> Solution: WA=Red, NT=Green, SA=Blue, Q=Red, NSW=Green, V=Red, T=যেকোনো।
> **অন্য উদাহরণ:** Sudoku, N-Queens, class routine/exam schedule (একই সময়ে একই room-এ দুই class নয়)।

সমাধান কৌশল

- **Backtracking search:** একটা একটা variable-এ মান বসাও; constraint ভাঙলে পিছিয়ে এসে অন্য মান।
- **Forward checking:** মান বসানোর পর প্রতিবেশীদের domain থেকে অসঙ্গত মান বাদ। কোনো domain ফাঁকা হলে সাথে সাথে backtrack।
- **Arc consistency (AC-3):** প্রতিটা constraint-এর জন্য এক পাশের প্রতিটা মানের অন্য পাশে সামঞ্জস্যপূর্ণ মান আছে কিনা নিশ্চিত করা; না থাকলে সেই মান বাদ।

- **Heuristics: MRV** (সবচেয়ে কম বৈধ মানওয়ালা variable আগে), **Degree** (সবচেয়ে বেশি constraint-এ জড়িত variable আগে), **LCV** (যে মান প্রতিবেশীদের সবচেয়ে কম সীমিত করে সেটা আগে)।

Forward checking (ছোট উদাহরণ)
A, B, C একে অপরের থেকে আলাদা হবে, Domain {1,2}। A=1 বসালে B,C-র domain হয় {2}। B=2 বসালে C-র domain ফাঁকা {} → সাথে সাথে বুঝে গেলাম এই পথ ব্যর্থ, backtrack। (আসলে এই সমস্যার সমাধান নেই, কারণ ৩টা আলাদা মান দরকার কিন্তু মাত্র ২টা আছে।)

Exam Tip
CBR-এ 4R cycle-এর diagram (বৃত্তাকার তীর) আঁকো। CSP-তে সবসময় map coloring বা N-Queens দিয়ে Variables/Domain/Constraints আলাদা করে লেখো, তারপর backtracking + MRV।

11. NLP & NLP using Generative AI

Pipeline • Embeddings • Transformer • LLM applications

১১.১ NLP কী?

Natural Language Processing = মানুষের ভাষা (বাংলা, English) কম্পিউটারকে বুঝতে ও তৈরি করতে শেখানো। Natural Language Understanding (NLU) + Natural Language Generation (NLG)।

১১.২ Classical NLP Pipeline

ধাপ	কাজ	উদাহরণ
Tokenization	বাক্যকে শব্দ/টুকরোয় ভাঙা	"আমি ভাত খাই" → [আমি, ভাত, খাই]
Stop-word removal	অপ্রয়োজনীয় শব্দ বাদ	the, is, এবং, যে
Stemming / Lemmatization	মূল রূপে আনা	running → run, better → good
POS tagging	শব্দের ব্যাকরণগত শ্রেণি	খাই = Verb
NER	নাম, স্থান, তারিখ শনাক্ত	"রহিম ঢাকায় গেছে" → Person, Location
Parsing	বাক্যের গঠন বিশ্লেষণ	Subject–Verb–Object
Vectorization	সংখ্যায় রূপান্তর: BoW, TF-IDF, Word2Vec/GloVe	"king − man + woman ≈ queen"

Tasks: sentiment analysis, text classification, machine translation, summarization, question answering, speech recognition, chatbot।

১১.৩ Transformer ও Modern NLP

- **RNN/LSTM** শব্দ একটা একটা করে পড়ে → ধীর, দীর্ঘ নির্ভরতায় দুর্বল।
- **Transformer** (2017, "Attention Is All You Need"): **self-attention** দিয়ে বাক্যের প্রতিটা শব্দ অন্য সব শব্দের দিকে একসাথে তাকায়, তাই parallel ও দীর্ঘ context ভালো বোঝে।
- **BERT** (encoder, দুই দিক দেখে বোঝে → classification, NER ভালো) বনাম **GPT** (decoder, পরের token predict করে → text generation ভালো)।

Self-attention সহজে
"ব্যাংকের পাড়ে বসে নদী দেখলাম" এবং "সে ব্যাংকে টাকা রাখল": "ব্যাংক" শব্দের অর্থ আশেপাশের শব্দ (নদী / টাকা) দেখে ঠিক হয়। Attention এই "কোন শব্দ কোন শব্দের সাথে সম্পর্কিত" তার weight হিসাব করে।

১১.৪ Generative AI দিয়ে NLP

আগে প্রতিটা task-এর জন্য আলাদা model লাগত। এখন একটা LLM **prompt** দিয়েই সব করে: zero-shot / few-shot।

Task	পুরোনো পদ্ধতি	GenAI/LLM পদ্ধতি
Sentiment	Labeled data দিয়ে classifier train	Prompt: "এই review ইতিবাচক না নেতিবাচক?"
Summarization	Extractive (বাক্য বেছে নেওয়া)	Abstractive (নিজের ভাষায় সংক্ষেপ)
Translation	Statistical / Seq2Seq	LLM translation, context ও tone বজায় রেখে
QA / Chatbot	Rule/retrieval	LLM + **RAG** (নিজের document থেকে তথ্য এনে উত্তর)

Scenario: ই-কমার্সে বাংলা review analysis
Review: "প্রোডাক্টটা দেখতে সুন্দর কিন্তু ডেলিভারি অনেক দেরিতে এসেছে।" Prompt: "এই review থেকে sentiment ও aspect-wise মতামত JSON-এ দাও।" Output: `{"product":"positive","delivery":"negative","overall":"mixed"}`। এতে আলাদা করে labeled data বা model train ছাড়াই কাজ হয়। সাথে LLM দিয়ে ক্ষুব্ধ গ্রাহককে auto-reply draft করা যায়।

Techniques: prompt engineering, fine-tuning (LoRA), RAG, instruction tuning, RLHF। **Challenges:** hallucination, বাংলার মতো low-resource ভাষায় কম data, bias, privacy। **Evaluation:** BLEU (translation), ROUGE (summary), F1/accuracy (classification), perplexity।

Exam Tip
Pipeline ধাপগুলো একটা বাংলা বাক্য দিয়ে দেখাও। "NLP vs GenAI-based NLP" তুলনা table লিখলে ভালো। BERT vs GPT পার্থক্য প্রায়ই আসে।

12. Quantum Computing vs Generative AI

তুলনা • সীমাবদ্ধতা • একসাথে কাজ (Synergy)

১২.১ মৌলিক পার্থক্য

দুটো সম্পূর্ণ আলাদা স্তরের প্রযুক্তি: **Quantum computing = নতুন ধরনের hardware ও computation model** (কীভাবে গণনা হয়); **Generative AI = software/algorithm যা data থেকে শিখে নতুন content বানায়** (কী করা হয়)। তাই এরা প্রতিদ্বন্দ্বী নয়, বরং ভিন্ন সমস্যার জন্য।

দিক	Quantum Computing	Generative AI
মূল ভিত্তি	Quantum physics (qubit, superposition, entanglement)	Statistics + deep learning (Transformer, Diffusion)
Hardware	Superconducting/ion trap, প্রায় শূন্য তাপমাত্রা (~15 mK), বিশেষ lab	সাধারণ GPU/TPU, cloud-এ সহজলভ্য
প্রধান উদ্দেশ্য	নির্দিষ্ট কঠিন সমস্যা দ্রুত সমাধান (factoring, search, simulation, optimization)	Text, image, code, audio তৈরি
Output প্রকৃতি	Probabilistic, কিন্তু গাণিতিকভাবে নির্ভুল উত্তর বের করার জন্য design করা	Probabilistic, অনুমানভিত্তিক; ভুল (hallucination) সম্ভব
Data প্রয়োজন	বড় training data লাগে না (algorithm-ভিত্তিক)	বিপুল training data অপরিহার্য
Speedup	কিছু সমস্যায় exponential/quadratic	Speedup নয়, নতুন ক্ষমতা (creativity, automation)
পরিপক্কতা (Maturity)	প্রাথমিক পর্যায়: noisy, qubit কম (NISQ era), error correction চলমান	ব্যাপক ব্যবহৃত, দৈনন্দিন product
সমস্যা	Decoherence, noise, scalability, খরচ	Hallucination, bias, বিপুল energy, copyright
উদাহরণ	Shor, Grover, VQE, QAOA	ChatGPT, Stable Diffusion, Copilot

Scenario: নতুন ওষুধ আবিষ্কার (Drug Discovery)

Quantum: অণুর আচরণ simulate করে (অণু নিজেই quantum system, তাই quantum computer স্বাভাবিকভাবে ভালো)। এতে বন্ধন শক্তি নির্ভুলভাবে বের করা যায়, যা classical computer-এ খুব ব্যয়বহুল।
GenAI: লক্ষ লক্ষ সম্ভাব্য নতুন অণুর structure তৈরি (generative chemistry) ও সবচেয়ে সম্ভাবনাময় গুলো বাছাই।
একসাথে: GenAI প্রার্থী অণু বানায় → Quantum সিমুলেশন সেগুলো যাচাই করে → সেরা প্রার্থী lab-এ।

১২.২ কীভাবে একে অপরকে সাহায্য করে (Synergy)

Quantum → AI	AI → Quantum
Quantum Machine Learning (QML): কিছু ML কাজ (kernel, optimization, sampling) দ্রুত করার সম্ভাবনা	Quantum **error correction** decode করতে neural network
Quantum generative models: QGAN, Quantum Boltzmann machine, QCBM	Quantum circuit design/optimization ও compiler
AI training-এর optimization সমস্যা (QAOA)	LLM দিয়ে quantum code (Qiskit) লেখা ও ব্যাখ্যা
Grover-ভিত্তিক দ্রুত search/sampling	Quantum hardware calibration ও noise model করা

গুরুত্বপূর্ণ বাস্তবতা

আজকের quantum computer দিয়ে বড় GenAI model (যেমন billions parameter-এর LLM) train করা সম্ভব নয়: qubit সংখ্যা কম, noise বেশি, বড় classical data quantum state-এ লোড করাও কঠিন। তাই বর্তমানে মূলত **hybrid** পদ্ধতি: বেশিরভাগ কাজ classical GPU-তে, শুধু নির্দিষ্ট অংশ quantum processor-এ। দীর্ঘমেয়াদে (fault-tolerant quantum computer) সম্ভাবনা আছে, তবে এখনও গবেষণার স্তরে।

১২.৩ নিরাপত্তার দিক

Quantum (Shor) RSA/ECC ভাঙতে পারে → **Post-Quantum Cryptography** (lattice-based, NIST standard যেমন ML-KEM/Kyber) দরকার; "harvest now, decrypt later" ঝুঁকি। **GenAI** ঝুঁকি: deepfake, phishing email স্বয়ংক্রিয়ভাবে বানানো, prompt injection। দুটোই cyber security-র নতুন চ্যালেঞ্জ (Topic 3-এর সাথে যুক্ত করে লেখা যায়)।

> **Exam Tip (এই topic তোমার teacher-এর নিজের কাজ, তাই খুব সম্ভাবনা আছে)**
> উত্তরের কাঠামো: ১) দুটোর সংজ্ঞা (১ লাইন করে) → ২) তুলনার table (উপরেরটা) → ৩) drug discovery scenario → ৪) Synergy (QML, AI for quantum) → ৫) Conclusion: "Quantum ও GenAI পরস্পরের বিকল্প নয়, পরিপূরক। Quantum বিশেষ কঠিন গণনায় শক্তিশালী, GenAI বিষয়বস্তু তৈরি ও automation-এ; ভবিষ্যতে hybrid system সবচেয়ে কার্যকর।"

Last-Minute Cheat Sheet

পরীক্ষার আগের ১৫ মিনিটে এটাই পড়ো

#	Topic	মনে রাখার মূল পয়েন্ট
1	Quantum	Qubit $\alpha\|0\rangle+\beta\|1\rangle$; Grover $O(\sqrt{N})$, oracle+diffusion, N=4 → 1 iteration; Shor = period finding + QFT, 15=3×5 (a=7, r=4); RSA ঝুঁকি
2	Blockchain	Block = prev hash + Merkle root + nonce; PoW vs PoS; immutable; smart contract; 51% attack
3	Cyber Security	CIA triad; phishing, SQLi (' OR '1'='1), DDoS, MITM; AES/RSA/SHA; MFA, firewall, IDS
4	LLM in SE	Codegen, test gen, review, repair; RAG, prompt; pass@k; hallucination, security, license
5	LLM/VLLM in DS	Text-to-SQL, cleaning, EDA; VLM = image+text (chart/OCR); vLLM = PagedAttention serving
6	CV & GenAI	Canny/SIFT/HOG; CNN, ResNet, YOLO, U-Net, ViT; GAN (G vs D), VAE, Diffusion
7	Ontology & DM	Class/Instance/Property/Axiom, OWL; KDD; Apriori: support, confidence, lift
8	KR & Frames	4 KR properties; frame = slots+facets+inheritance+default+demons; Vehicle→Car→MyToyota
9	SemNet, Rules, SemWeb	is-a inheritance; IF-THEN, forward vs backward chaining; RDF triple, OWL, SPARQL, layer cake
10	CBR & CSP	Retrieve-Reuse-Revise-Retain; CSP = X, D, C; map coloring; backtracking, forward checking, AC-3, MRV
11	NLP & GenAI	Tokenize→stem→POS→NER→parse; TF-IDF/Word2Vec; Transformer self-attention; BERT vs GPT; RAG
12	Quantum vs GenAI	Hardware/physics vs data/software; table; drug discovery; QML ↔ AI for quantum; hybrid; PQC

শেষ কথা

প্রতিটা উত্তরে: **Definition → Points → Example → Pros/Cons → Diagram/Table**। বাংলা-English মিশিয়ে লিখলে technical term English-এই রাখো (যেমন superposition, inference engine), ব্যাখ্যাটা বাংলায় দাও। সবার জন্য শুভকামনা, ভালো করে দিও পরীক্ষা!